

AI EXTENSION TEST SCRIPT GENERATOR

Week3 – Day1 Assignment



JUNE 25, 2026
ANITHA PRAJISH

Contents

1. Title.....	2
2. Objective	2
3. Extension Details.....	2
4. Files / Structure.....	2
5. Working / Flow.....	2
6. Permissions Used.....	2
7. Test / Observation.....	2
8. Output / Screenshot.....	3
9. Implementation of Selenium-C#.....	4
Ask Prompt.....	4
Response:	4
Agent Prompt	12
Browser Extension Output.....	12
10. Conclusion.....	17

1. Title

Browser Extension for Automated Test Script Generation

2. Objective

The objective of this assignment is to analyze a browser extension that generates automated test scripts based on the user's selected programming language and supported browser engine. Currently, the extension supports Selenium-Java and Playwright-TypeScript, with the goal of extending its functionality by implementing support for either Selenium-C# or Playwright-C#.

3. Extension Details

- Extension Name: ai-extension
- Browser: Chrome

4. Files / Structure

manifest.json – Main configuration file
bg.js – Background tasks
content.js – Webpage interaction
panel.html – Popup UI
popup.js – Handles popup actions

5. Working / Flow

1. Install extension
2. Browser reads manifest.json
3. Enter details in Settings. (Language, Browser, LLM Provider, Model & API key)
4. Background script starts
5. Content script interacts with webpage
6. Open Leaftaps.com. In the extension, choose Inspect and select the required fields. Then select Feature File and Page Class, and click Generate.
7. Output is displayed in chat box

6. Permissions Used

- Active Tab
- Website URL access

7. Test / Observation

- Extension loads successfully
- Popup opens correctly
- Expected action executes without errors

8. Output / Screenshot

The screenshot displays the Testleaf CRM/SAF eCommerce interface. The main content area is the 'Create Contact' form, which is divided into several sections: 'Contact Information' (First Name, Last Name, Salutation, Birth Date, Department, Title, Preferred Currency, Initial Account, Description, Important Note), 'Contact Information' (Country Code, Area Code, Extension, E-Mail Address, Phone Number, Person to Ask For), and 'Primary Address' (To Name, Address Line 1, City, Zip/Postal Code, Attention Name, Address Line 2, State/Province, Country). A 'Create Contact' button is at the bottom right of the form.

On the right side, there is an 'AI Browser Extension for Code Generation' sidebar. It features a 'Reset Chat' button, a 'Feature: Create Contact' section, a 'Scenario Outline: Successful contact' section, and a 'Provide additional information' section. The sidebar also includes 'Inspect' and 'Generate' buttons.

The screenshot shows the 'AI Browser Extension for Code Generation' settings panel. The panel is titled 'Code Generator' and 'Settings'. It contains several configuration options:

- Supported Language:** A dropdown menu with 'Java' selected.
- Browser Engine:** A dropdown menu with 'Selenium' selected.
- Select LLM Provider:** A dropdown menu with 'Groq' selected.
- Select Model:** A dropdown menu with 'llama-3.3-70b-versatile' selected.
- Groq API Key:** A text input field with a placeholder '.....'.

9. Implementation of Selenium-C#

Opened the ai-extension project in Visual Studio Code, accessed GitHub Copilot, selected Ask mode with Claude Haiku 4.5, and entered the prompt:

Ask Prompt

"Analyze the existing codebase that supports Selenium-Java and Playwright-TypeScript for test script generation. Add Selenium-C# support. Review the project and identify all files responsible for code generation, language/browser selection, templates, configurations, and UI logic. Provide a file-by-file breakdown of what needs to be updated, any new files required, reusable code from Selenium-Java, dependencies needed, and a step-by-step implementation plan."

Response:

Based on my comprehensive analysis of your codebase, here's a complete breakdown:

ARCHITECTURE OVERVIEW

Your extension uses a **prompt-driven, modular code generation system** where:

- DOM content is captured via inspector
- Language/Browser/Mode selections determine which prompts are used
- LLM APIs generate code using templated prompts
- Prism highlights the generated code

FILE-BY-FILE BREAKDOWN

Critical Files for Code Generation:

prompts.js

Responsibility: Template repository for all code generation

- `DEFAULT\_PROMPTS` object: Contains 6 prompt templates
 - `SELENIUM\_JAVA\_PAGE\_ONLY` - Page object class only
 - `CUCUMBER\_ONLY` - Feature file only
 - `CUCUMBER\_WITH\_SELENIUM\_JAVA\_STEPS` - Combined feature + step definitions
 - `PLAYWRIGHT\_TYPESCRIPT\_POM` - Page object model
 - `PLAYWRIGHT\_TYPESCRIPT\_SPEC` - Test spec only
 - `PLAYWRIGHT\_TYPESCRIPT\_POM\_AND\_SPEC` - Combined
- `getPrompt(promptKey, variables)` - Template variable substitution function
- `CODE\_GENERATOR\_TYPES` - Enum of all supported generation types

chat.js

Responsibility: Main logic orchestrator

- **Language/Browser selection:** `this.languageBindingSelect` (id="languageBinding"), `this.browserEngineSelect` (id="browserEngine")
- **Generation mode:** Checkboxes with `name="javaGenerationMode"` (hardcoded name - a refactoring opportunity)
- **Critical method** `getPromptKeys(language, engine)`:

- Routes to correct prompts based on language + engine + generation mode
- Helper methods: `isJavaSelenium()`, `isTypeScriptPlaywright()`, `isCSharpSelenium()` (stubbed)
- Logic: "Feature only" vs "Page only" vs "Both"
- **Message handling**: `sendMessage()` calls LLM with populated prompts
- **Syntax highlighting**: Prism language mapping including `cs`: 'csharp'

panel.html

Responsibility: UI controls

- **Language dropdown** (id="languageBinding"): java, csharp, py, ts
- **Browser dropdown** (id="browserEngine"): selenium, playwright, cypress, puppeteer
- **Generation mode checkboxes** (name="javaGenerationMode"): Feature File, Page Class
-  **Note**: Checkbox name is hardcoded for Java - works for other languages by accident

groq-api.js, openai-api.js, testleaf-api.js

Responsibility: LLM provider wrappers

- Generic message sending - already support any language
- No language-specific changes needed
- Return: `{ content, usage: { input\_tokens, output\_tokens } }`

appConfig.js

Responsibility: Color/styling configuration

- No language-specific logic needed

Other files

- popup.js - Settings tab logic (language/model selection)
- tabs.js - Tab switching (minimal changes)
- bg.js - Background service worker (no changes)
- prism - Syntax highlighting library

WHAT NEEDS TO BE UPDATED

1. NEW PROMPTS (in prompts.js)

Add these C# prompts following the existing pattern:

```
```\javascript
```

```
SELENIUM_CSHARP_PAGE_ONLY: `
```

Instructions:

- Generate ONLY a Selenium C# Page Object Class (no test code).
- Use NUnit or MSTest framework conventions.
- Use C# async/await patterns.
- Include XML documentation comments.
- Use Selenium 4.x compatible imports.
- Use meaningful method names following PascalCase.
- Do NOT include test code or explanations.

```
...
```

SELENIUM\_CSHARP\_WITH\_NUNIT: `

Instructions:

- Generate BOTH:

1. A Selenium C# Page Object Class.
2. An NUnit test class using the POM.

- Page Object: Properties and methods for page interactions.

- Test Class: [TestFixture], [Test] attributes with realistic scenarios.

...

`

...

### \*\*2. LOGIC UPDATES (in chat.js)\*\*

**\*\*Implement `isCSharpSelenium()` method\*\*** (currently stubbed):

```
```javascript
isCSharpSelenium(language, engine) {
  return language === 'csharp' && engine === 'selenium';
}
```
```

**\*\*Update `getPromptKeys()` method\*\*** to handle C#:

```
```javascript
if (isFeatureChecked && isPageChecked) {
  if (this.isJavaSelenium(lang, eng)) {
    promptKeys.push('CUCUMBER_WITH_SELENIUM_JAVA_STEPS');
  } else if (this.isCSharpSelenium(lang, eng)) {
    promptKeys.push('SELENIUM_CSHARP_WITH_NUNIT'); // NEW
  } else if (this.isTypeScriptPlaywright(lang, eng)) {
    promptKeys.push('PLAYWRIGHT_TYPESCRIPT_POM_AND_SPEC');
  }
  // ...
}
```
```

### \*\*3. SYNTAX HIGHLIGHTING\*\*

Add Prism C# language support to panel.html:

```
```html
<script
src="https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.8.0/languages/csharp.min.js">
</script>
```
```

Or include locally in `/lib/prism/prism-csharp.js`

Update language mapping in chat.js:

```
```javascript
const languageMap = {
  'feature': 'gherkin',

```

```

    'cucumber': 'gherkin',
    'bdd': 'gherkin',
    'js': 'javascript',
    'ts': 'typescript',
    'py': 'python',
    'cs': 'csharp',
    'csharp': 'csharp' // ADD THIS
  };
  ...

```

4. OPTIONAL: UI IMPROVEMENTS

Rename hardcoded checkbox names in panel.html:

```

```html
<!-- BEFORE -->
<input type="checkbox" name="javaGenerationMode" value="FEATURE" />

<!-- AFTER -->
<input type="checkbox" name="generationMode" value="FEATURE" />
```

```

Then update selector in chat.js:

```

```javascript
// BEFORE
const checkboxes =
Array.from(document.querySelectorAll('input[name="javaGenerationMode"]:checked'));

// AFTER
const checkboxes =
Array.from(document.querySelectorAll('input[name="generationMode"]:checked'));
```

```

5. UPDATE CODE_GENERATOR_TYPES ENUM

In prompts.js:

```

```javascript
export const CODE_GENERATOR_TYPES = {
 SELENIUM_JAVA_PAGE_ONLY: 'Selenium-Java-Page-Only',
 SELENIUM_CSHARP_PAGE_ONLY: 'Selenium-CSharp-Page-Only', // NEW
 SELENIUM_CSHARP_WITH_NUNIT: 'Selenium-CSharp-With-NUnit', // NEW
 CUCUMBER_ONLY: 'Cucumber-Only',
 // ... rest unchanged
};
```

```

REUSABLE CODE FROM SELENIUM-JAVA

| Component | Reusable Aspect | Adaptation for C# |


```

|-----|-----|-----|
| **Prompt Structure** | ICE POT format, DOM template | Replace Java syntax with C#
syntax |
| **DOM Processing** | stripExtraOptions, DOMParser | No changes - DOM processing is
language-agnostic |
| **Generation Mode Logic** | "Feature + Page" routing | Reuse, just add C#-specific prompt
key |
| **Message Flow** | UI → LLM → Parse → Display | 100% reusable, language-agnostic |
| **API Wrappers** | Groq/OpenAI/Testleaf clients | 100% reusable - they're provider-
agnostic |
| **Storage** | Chrome storage for settings | 100% reusable |

```

****NEW DEPENDENCIES REQUIRED****

```

Dependency	Purpose	Where	Size
**Prism C# Plugin**	Syntax highlighting for C# code	`/lib/prism/prism-csharp.js`	
~5KB			
**NUnit/MSTest Template Knowledge**	C# testing framework conventions	In	
prompts.js (no external dep) | N/A |

```

****STEP-BY-STEP IMPLEMENTATION PLAN****

****Step 1: Add C# Prompt Templates**** (5 min)

- [] Open prompts.js
- [] Add `SELENIUM\_CSHARP\_PAGE\_ONLY` prompt (copy Java template, adapt syntax)
- [] Add `SELENIUM\_CSHARP\_WITH\_NUNIT` prompt (copy Cucumber+Steps template, adapt)
- [] Update `CODE\_GENERATOR\_TYPES` enum to include new C# types

****Example prompt template structure**:**

```
```javascript
```

```
SELENIUM_CSHARP_PAGE_ONLY: `
```

```
 Instructions:
```

- Generate ONLY a Selenium C# Page Object Class (no test code).
- Use Selenium 4.x compatible imports (OpenQA.Selenium).
- Add XML documentation comments (///).
- Use C# async/await patterns where applicable.
- Use meaningful method names following PascalCase convention.
- Do NOT include explanations or test code.

```
Context:
```

```
DOM:
```

```
\\`\\`html
```

```
\\${domContent}
```

```
\\`\\`
```

Example:

```
\\\`csharp
using OpenQA.Selenium;
using OpenQA.Selenium.Support.UI;
```

```
/// <summary>
/// Page Object for Login Page
/// </summary>
public class LoginPage {
 // Add methods as per the DOM
}
\\\`
```

Persona:

- Audience: C# automation engineers building maintainable Page Object Models.

Output Format:

- A single C# class inside a `\\\`csharp\\\` block.

Tone:

- Clean, maintainable, enterprise-ready.

...

### \*\*Step 2: Implement C# Detection Logic\*\* (2 min)

- [ ] Open chat.js
- [ ] Uncomment/implement `isCSharpSelenium()` method (it's already stubbed!)
- [ ] Update `getPromptKeys()` method to handle C#/Selenium combinations

### \*\*Step 3: Add Syntax Highlighting Support\*\* (3 min)

- [ ] Option A (CDN): Update panel.html to include Prism C#:

```
``html
<script
src="https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.8.0/languages/csharp.min.js">
</script>
``
```

- [ ] Option B (Local): Download `prism-csharp.js` to prism and reference it
- [ ] Add language mapping in chat.js languageMap object

### \*\*Step 4: (Optional) Fix UI Checkbox Names\*\* (5 min)

- [ ] Rename `javaGenerationMode` to `generationMode` in panel.html
- [ ] Update selector in chat.js
- [ ] This makes the code more generic and maintainable for future languages

### \*\*Step 5: Test End-to-End\*\* (10 min)

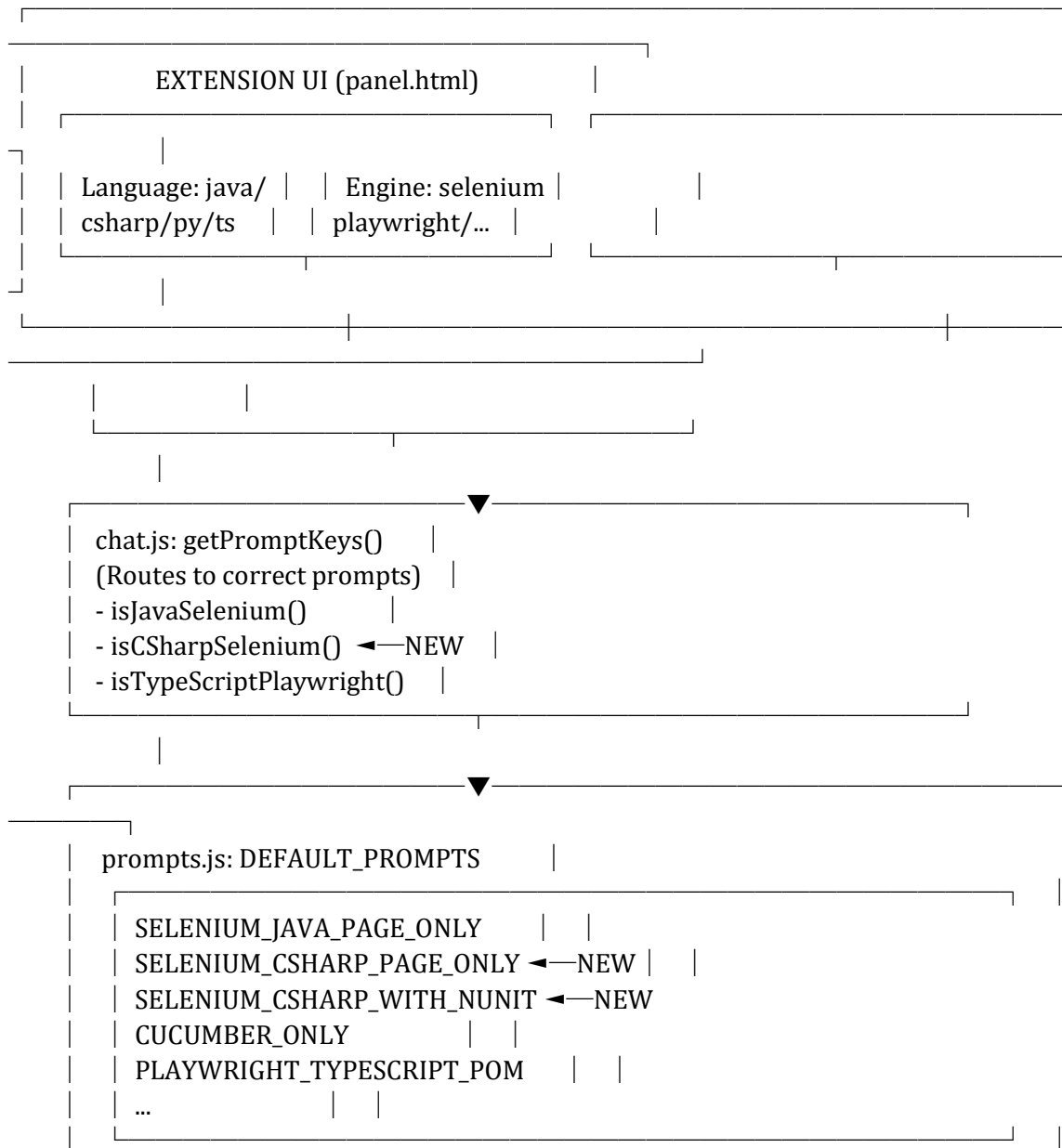
- [ ] Select "C#" + "Selenium" in Settings
- [ ] Select generation modes (Feature File / Page Class)
- [ ] Inspect DOM on a sample webpage
- [ ] Click "Generate"

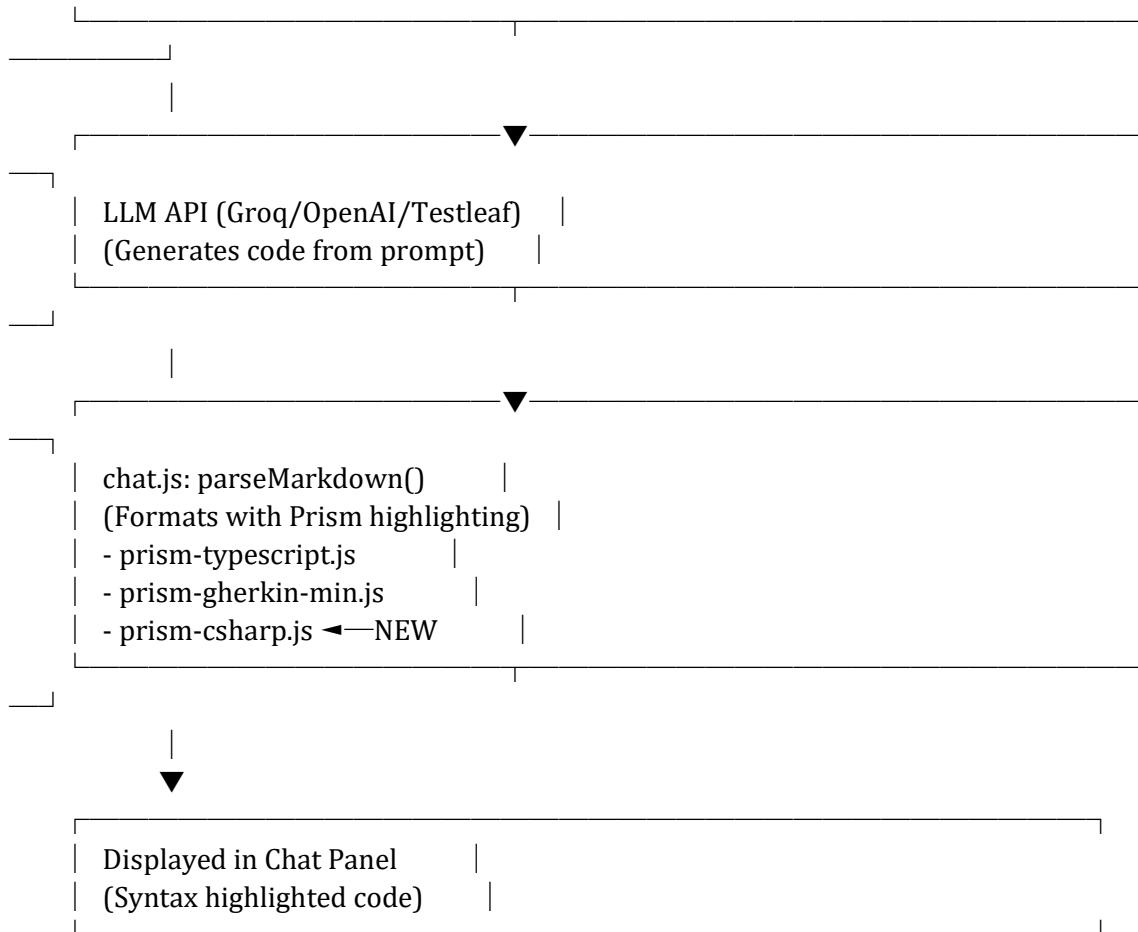
- [ ] Verify:
- [ ] Correct prompts are selected
- [ ] C# code is generated correctly
- [ ] Syntax highlighting works
- [ ] Copy button works
- [ ] Token counts display

---

## ## \*\*ARCHITECTURE DIAGRAM\*\*

---





...

---

## ## \*\*KEY CONSIDERATIONS\*\*

1. **Checkbox Names**: Currently hardcoded as `javaGenerationMode` but works for all languages. Good QA test to fix this during implementation.
2. **Prompt Quality**: C# prompts should follow .NET conventions:
  - Use `OpenQA.Selenium` namespace (not `Selenium`)
  - Use `WebDriverWait` with `TimeSpan` (not Java's `Duration`)
  - Consider NUnit vs MSTest (NUnit is more Cucumber-friendly)
  - XML documentation ( `///` ) instead of JavaDoc
3. **Syntax Highlighting**: Prism includes C# out-of-the-box or via CDN - no custom work needed beyond including the library.
4. **Testing Frameworks**: For C#, consider:
  - **NUnit** (BDD-friendly, Cucumber-like, recommended)
  - **MSTest** (Microsoft standard)
  - **xUnit** (modern alternative)

5. **Language Mapping**: Already has `cs: 'csharp'` mapping in chat.js, so add full coverage.

---

This implementation is **~15-20 minutes of work** with most of the infrastructure already in place for supporting new languages!

### Agent Prompt

"Implement Selenium-C# support in the existing browser extension that currently generates test scripts for Selenium-Java and Playwright-TypeScript. Review and update all required files including language selection logic, script generation templates, UI options, configurations, dependencies, and code generation modules. Reuse the existing Selenium-Java implementation wherever possible. Perform the implementation step by step, explain each change, identify the files being updated, and wait for my approval before proceeding to the next step."

### Browser Extension Output

Feature: Create Contact

Scenario Outline: Successful contact creation with valid first and last names

Given I open the create contact page  
When I type "<firstName>" into the First Name field  
And I type "<lastName>" into the Last Name field  
And I click the Create Contact button  
Then I should see the contact created successfully

Examples:

	firstName		lastName	
	Ramesh		Kumar	
	Suresh		Patel	
	Rohan		Sharma	

Scenario Outline: Unsuccessful contact creation with empty first name

Given I open the create contact page  
When I type "<firstName>" into the First Name field  
And I type "Kumar" into the Last Name field  
And I click the Create Contact button  
Then I should see an error message for empty first name

Examples:

	firstName	

Scenario Outline: Unsuccessful contact creation with empty last name

Given I open the create contact page

When I type "Ramesh" into the First Name field

And I type "<lastName>" into the Last Name field

And I click the Create Contact button

Then I should see an error message for empty last name

Examples:

| lastName |

|        |

|        |

Scenario Outline: Unsuccessful contact creation with special characters in first name

Given I open the create contact page

When I type "<firstName>" into the First Name field

And I type "Kumar" into the Last Name field

And I click the Create Contact button

Then I should see an error message for invalid first name

Examples:

| firstName |

| Ramesh! |

| Suresh@ |

Scenario Outline: Unsuccessful contact creation with special characters in last name

Given I open the create contact page

When I type "Ramesh" into the First Name field

And I type "<lastName>" into the Last Name field

And I click the Create Contact button

Then I should see an error message for invalid last name

Examples:

| lastName |

| Kumar! |

| Patel@ |

Scenario Outline: Unsuccessful contact creation with numbers in first name

Given I open the create contact page

When I type "<firstName>" into the First Name field

And I type "Kumar" into the Last Name field

And I click the Create Contact button

Then I should see an error message for invalid first name

Examples:

| firstName |

| Ramesh1 |

| Suresh2 |

Scenario Outline: Unsuccessful contact creation with numbers in last name

Given I open the create contact page

When I type "Ramesh" into the First Name field  
And I type "<lastName>" into the Last Name field  
And I click the Create Contact button  
Then I should see an error message for invalid last name

Examples:

```
| lastName |
| Kumar1 |
| Patel2 |
```

Copy

```
using OpenQA.Selenium;
using OpenQA.Selenium.Chrome;
using OpenQA.Selenium.Support.UI;
using System;

/// <summary>
/// Page Object for Create Lead Form
/// </summary>
public class CreateLeadFormPage {
 private IWebDriver driver;
 private WebDriverWait wait;

 /// <summary>
 /// Locator for Company Name input field
 /// </summary>
 private By CompanyNameInput => By.Id("createLeadForm_companyName");

 /// <summary>
 /// Locator for Parent Party Id input field
 /// </summary>
 private By ParentPartyIdInput => By.Id("createLeadForm_parentPartyId");

 /// <summary>
 /// Locator for First Name input field
 /// </summary>
 private By FirstNameInput => By.Id("createLeadForm_firstName");

 /// <summary>
 /// Locator for Create Lead submit button
 /// </summary>
 private By CreateLeadButton => By.Id("ext-gen529");

 /// <summary>
 /// Initializes a new instance of the <see cref="CreateLeadFormPage"/> class.
 /// </summary>
 /// <param name="driver">The Selenium WebDriver instance.</param>
 public CreateLeadFormPage(IWebDriver driver) {
 this.driver = driver;
```

```

 this.wait = new WebDriverWait(driver, TimeSpan.FromSeconds(10));
 }

 /// <summary>
 /// Navigates to the Create Lead Form page.
 /// </summary>
 public void GoTo() {
 driver.Navigate().GoToUrl("https://leaptaps.com/crmsfa/control/createLeadForm");
 }

 /// <summary>
 /// Enters the company name in the company name field.
 /// </summary>
 /// <param name="companyName">The company name to enter.</param>
 public void EnterCompanyName(string companyName) {
 IWebElement element =
wait.Until(ExpectedConditions.ElementToBeClickable(CompanyNameInput));
 element.SendKeys(companyName);
 }

 /// <summary>
 /// Enters the parent party id in the parent party id field.
 /// </summary>
 /// <param name="parentPartyId">The parent party id to enter.</param>
 public void EnterParentPartyId(string parentPartyId) {
 IWebElement element =
wait.Until(ExpectedConditions.ElementToBeClickable(ParentPartyIdInput));
 element.SendKeys(parentPartyId);
 }

 /// <summary>
 /// Enters the first name in the first name field.
 /// </summary>
 /// <param name="firstName">The first name to enter.</param>
 public void EnterFirstName(string firstName) {
 IWebElement element =
wait.Until(ExpectedConditions.ElementToBeClickable(FirstNameInput));
 element.SendKeys(firstName);
 }

 /// <summary>
 /// Clicks the Create Lead submit button.
 /// </summary>
 public void ClickCreateLead() {
 driver.FindElement(CreateLeadButton).Click();
 }
}

using NUnit.Framework;
using OpenQA.Selenium;

```



```

using OpenQA.Selenium.Chrome;
using System;

/// <summary>
/// Test class for Create Lead functionality
/// </summary>
[TestFixture]
public class CreateLeadTests {
 private IWebDriver driver;
 private CreateLeadFormPage createLeadFormPage;

 [SetUp]
 public void SetUp() {
 driver = new ChromeDriver();
 driver.Manage().Window.Maximize();
 createLeadFormPage = new CreateLeadFormPage(driver);
 createLeadFormPage.GoTo();
 }

 [TearDown]
 public void TearDown() {
 if (driver != null) {
 driver.Quit();
 }
 }

 [Test]
 public void TestCreateLeadWithValidData() {
 createLeadFormPage.EnterCompanyName("ABC Corporation");
 createLeadFormPage.EnterParentPartyId("12345");
 createLeadFormPage.EnterFirstName("Rajesh Kumar");
 createLeadFormPage.ClickCreateLead();

 Assert.That(driver.Url, Does.Contain("viewLead"));
 }

 [Test]
 public void TestCreateLeadWithEmptyCompanyName() {
 createLeadFormPage.EnterParentPartyId("12345");
 createLeadFormPage.EnterFirstName("Rajesh Kumar");
 createLeadFormPage.ClickCreateLead();

 Assert.That(driver.PageSource, Does.Contain("Company Name"));
 }

 [Test]
 public void TestCreateLeadWithEmptyFirstName() {
 createLeadFormPage.EnterCompanyName("ABC Corporation");
 createLeadFormPage.EnterParentPartyId("12345");
 createLeadFormPage.ClickCreateLead();
 }
}

```

```
 Assert.That(driver.PageSource, Does.Contain("First Name"));
 }
}
```

## 10. Conclusion

This assignment helped in understanding browser extension architecture, project files, and how the extension interacts with web pages.